

CCDEVAP

Web Application Development

CSS

Selectors, Specificity, Properties

Outline

- What is CSS and its structure
- CSS selectors
- Specificity
- Common properties
- Hands-on exercise

HTML

allows to create a document with any content

CCAPDEV: Web Application Development

AY 2025-2026 Term 3

Schedule

Section	Days	Time	Instructor
S15	MH	0730-0900	Beredo, Jackylyn L.
S16	MH	1100-1230	Beredo, Jackylyn L.

Topics

1. HTML, CSS
2. JavaScript, jQuery
3. Node.js
4. Database
5. Application deployment

Teaching Methods

- Lectures
- Recitation
- Mini Challenges
- Machine Project

Request a consult

Name:

Section:

☐ S15 ☐ S16

Date: mm / dd / yyyy 

CSS

allows styling to the
HTML document

CCDEVAP: Web Application Development

AY 2025-2026 Term 3

Schedule

SECTION	DAYS	TIME	INSTRUCTOR
S15	MH	0730-0900	Beredo, Jackylyn L.
S16	MH	1100-1230	Beredo, Jackylyn L.

Topics

1. HTML, CSS
2. JavaScript, jQuery
3. Node.js
4. Database
5. Application deployment

Teaching Methods

- Lectures
- Recitation
- Mini Challenges
- Machine Project

Request a consult

Name:

Section:

☐ S15

☐ S16

Date:



Cascading Style Sheet (CSS)

- **Styling language** with a set of rules that tell the browser ***how*** to display your document content
- Separate from the content
- Simple text file with an extension of **.css**

Cascading

- Procedure that determines which style will apply to a certain element or section
- Avoids repetitive declaration of styles
- Uses inheritance
 - Properties that apply to children *cascade* down

Cascading

Style

Sheet

Style

- Determines the design of the page
- Customize attributes such as colors, margins, fonts...

Cascading



Style

Sheet

style.css ✕

```
1  root {
2    --primary-color: #006a4e; /* DLSU Green */
3    --text-color: #333;
4    --border-color: #ddd;
5    --bg-light: #f9f9f9;
6  }
7
8  body {
9    font-family: 'Segoe UI', Roboto, Helvetica, Arial, sans-serif;
10   line-height: 1.6;
11   color: var(--text-color);
12   max-width: 800px;
13   margin: 40px auto;
14   padding: 0 20px;
15   background-color: #fff;
16 }
17
18 /* Header & Logos */
19 .header-logos {
20   display: flex;
21   gap: 20px;
22   margin-bottom: 20px;
23 }
24
25 .header-logos img {
26   height: 60px;
27   width: auto;
28   filter: drop-shadow(0 2px 4px rgba(0,0,0,0.1));
29 }
30
31 h1 {
32   color: var(--primary-color);
33   font-size: 2.2rem;
34   margin-bottom: 5px;
35 }
```

Cascading Style Sheet

Using **CSS** we can create containers and divisions to better organize content and make the page visually appealing.

CCDEVAP: Web Application Development

AY 2025-2026 Term 3

Schedule

SECTION	DAYS	TIME	INSTRUCTOR
S15	MH	0730-0900	Beredo, Jackylyn L.
S16	MH	1100-1230	Beredo, Jackylyn L.

Topics

1. HTML, CSS
2. JavaScript, jQuery
3. Node.js
4. Database
5. Application deployment

Teaching Methods

- Lectures
- Recitation
- Mini Challenges
- Machine Project

Request a consult

Name:

Section:

☐ S15

☐ S16

Date:

mm / dd / yyyy



CCAPDEV: Web Application Development

AY 2025-2026 Term 3

Schedule

Section	Days	Time	Instructor
S15	MH	0730-0900	Beredo, Jackylyn L.
S16	MH	1100-1230	Beredo, Jackylyn L.

Topics

1. HTML, CSS
2. JavaScript, jQuery
3. Node.js
4. Database
5. Application deployment

Teaching Methods

- Lectures
- Recitation
- Mini Challenges
- Machine Project

Request a consult

Name:

Section:

☐ S15 ☐ S16

Date: mm / dd / yyyy



Advantages of CSS

- **Separates structure from presentation.** Use HTML for the structure of the page and use CSS to define the look and feel of the page.
- **Provides advanced control of presentation.** CSS has a much wider array of attributes than HTML, so you can give a far better look to your page using CSS than HTML attributes.
- **Reusability.** Styles for one HTML can be reused for different HTML pages. Define a style for each element and apply it to multiple pages.
- **Easy maintenance of multiple pages.** To make a global design change, you can simply change the style for a selector and all elements defined by the selector in all connected web pages will be updated automatically.
- **Faster page loading.** Using CSS, you do not need to rewrite HTML tag attributes every single time. One CSS rule for a tag applies to all occurrences of that tag. Less code means faster loading time.

CSS Structure

selector

```
{
```

property: **value;**

```
}
```

This is what we refer to as the CSS rule.

- A **selector** tag is a tag in which the style will be applied to
- A **property** is the attribute or characteristic of the HTML element you want to change.
- The **value** is assigned to the property you want to change

```
:root {
  --primary-color: #006a4e; /* DLSU Green */
  --text-color: #333;
  --border-color: #ddd;
  --bg-light: #f9f9f9;
}

body {
  font-family: 'Segoe UI', Roboto, Helvetica, Arial, sans-serif;
  line-height: 1.6;
  color: var(--text-color);
  max-width: 800px;
  margin: 40px auto;
  padding: 0 20px;
  background-color: #fff;
}

/* Header & Logos */
.header-logos {
  display: flex;
  gap: 20px;
  margin-bottom: 20px;
}

.header-logos img {
  height: 60px;
  width: auto;
  filter: drop-shadow(0 2px 4px rgba(0,0,0,0.1));
}

h1 {
  color: var(--primary-color);
  font-size: 2.2rem;
  margin-bottom: 5px;
}

h2 {
  border-left: 5px solid var(--primary-color);
  padding-left: 15px;
  margin-top: 30px;
  font-size: 1.5rem;
}

h1 + p {
  font-weight: bold;
  text-align: center;
}

.section-container {
  display: flex;
  gap: 20px;
  /* This aligns the sections side by side */
  /* Adds space between the two sections */
}
```

Recitation

- What are the 3 ways to integrate CSS in HTML?

3 Ways to use CSS

- External Style Sheet
- Internal Style Sheet
- Inline Style Sheet

External Style Sheet

- Saved in an external file with **.css** as the file extension
- Best used to control styling on multiple web pages
- Loaded through the **<head>** tag of the HTML file

```
<link rel="stylesheet" type="text/css" media="all" href="css/styles.css">
```

- **rel** - *Required*. Specifies the relationship between the current document and the linked document (*common values include icon and stylesheet*)
- **type** - Specifies the media type of the linked document (*common values include text/css and text/javascript*)
- **media** - Specifies on what kind of devices the linked document will be displayed
- **href** - *Required*. Specifies the location of the linked document

Internal Style Sheet

- Best used to control styling on a **single** web page
- Defined by the **<style>** tag within the **<head>** tag

```
<head>  
  <style type="text/css">  
    h1 {  
      color: #FF0000;  
    }  
  </style>  
</head>
```


Inline Style Sheet

- Used to override styles specific to an HTML element
- Directly loaded using the style attribute of the HTML tag
- *Highly discouraged*

```
<p style="color: #FF0000; font-weight: bold;">
```

This text is red!

```
</p>
```

General Precedence Rules

The *cascade* of the rules start from the **farthest** to the **closest** to the element.

1. User-agent (browser) default style sheets
2. External style sheets
3. Internal/Embedded style sheets
4. Inline style sheets

CSS Selectors

finding HTML tags to apply styles on

Five Categories

1. **Simple** selectors - select elements based on tag, id, class
2. **Combinator** selectors - select elements based on a specific relationship between them
3. **Pseudo-class** selectors - select elements based on a certain state
4. **Pseudo-element** selectors - select and style a part of an element
5. **Attribute** selectors - select elements based on an attribute or attribute value

CSS Structure

selector

```
{  
  property: value;  
}
```

This is what we refer to as the CSS rule.

- A **selector** tag is a tag in which the style will be applied to
- A **property** is the attribute or characteristic of the HTML element you want to change.
- The **value** is assigned to the property you want to change

```
:root {  
  --primary-color: #006a4e; /* DLSU Green */  
  --text-color: #333;  
  --border-color: #ddd;  
  --bg-light: #f9f9f9;  
}  
  
body {  
  font-family: 'Segoe UI', Roboto, Helvetica, Arial, sans-serif;  
  line-height: 1.6;  
  color: var(--text-color);  
  max-width: 800px;  
  margin: 40px auto;  
  padding: 0 20px;  
  background-color: #fff;  
}  
  
/* Header & Logos */  
.header-logos {  
  display: flex;  
  gap: 20px;  
  margin-bottom: 20px;  
}  
  
.header-logos img {  
  height: 60px;  
  width: auto;  
  filter: drop-shadow(0 2px 4px rgba(0,0,0,0.1));  
}  
  
h1 {  
  color: var(--primary-color);  
  font-size: 2.2rem;  
  margin-bottom: 5px;  
}  
  
h2 {  
  border-left: 5px solid var(--primary-color);  
  padding-left: 15px;  
  margin-top: 30px;  
  font-size: 1.5rem;  
}  
  
h1 + p {  
  font-weight: bold;  
  text-align: center;  
}  
  
.section-container {  
  display: flex; /* This aligns the sections side by side */  
  gap: 20px; /* Adds space between the two sections */  
}
```

Simple Selectors – Tag

HTML

```
<h3 class="comment">
    My name
</h3>
<p class="comment">
    Sample comment
</p>
```

CSS

```
h3 {
    color: blue;
}
```

- selects by tags such as <p>, <h3>, <a>, <div>

Simple Selectors – Multiple Tags

HTML

```
<h1>
    My name
</h1>
<p>
    Sample comment
</p>
<h3>
    End
</h3>
```

CSS

```
h3, p {
    font-weight: bold;
}
```

- multiple tags separated by comma (,)

Simple Selectors – ID

HTML

```
<div id="menu">
  <ul>
    <li>Home</li>
    <li>Profile</li>
  </ul>
</div>
<div>
  <p> hello </p>
</div>
```

CSS

```
#menu {
  background-color:black;
}
```

- ID name denoted by #

Simple Selectors – Multiple IDs

HTML

```
<div id="title">
    ...
</div>
<p id="about">
    ...
</p>
<div id="hello">
    <p> hello </p>
</div>
```

CSS

```
#title, #about {
    background-color:black;
}
```

- multiple ID names separated by comma (,)

Recitation

- What is the difference between tag ID and class?

Simple Selectors – Class

HTML

```
<h3 class="comment">
    My name
</h3>
<p class="comment">
    Sample comment
</p>
<div class="hello">
    <p> hello </p>
</div>
```

CSS

```
.comment {
    color: blue;
}
```

- class name denoted by a period (.) before the value

Simple Selectors – Multiple Classes

HTML

```
<p class="center bold">  
    Text in center and bold  
</p>  
<p class="center">  
    Text in center  
</p>
```

CSS

```
.center {  
    text-align: center;  
}  
  
.bold {  
    font-weight: bold;  
}
```

- multiple classes with different styles should be separated

Simple Selectors – Multiple Classes

HTML

```
<h3 class="name">
    My name
</h3>
<p class="comment">
    Sample comment
</p>
```

CSS

```
.name, .comment {
    font-weight: bold;
}
```

- multiple class names with same style separated by a comma (,)

Simple Selectors – Element + Class

HTML

```
<h3 class="comment">
    My name
</h3>
<p class="comment">
    Sample comment
</p>
```

CSS

```
p.comment {
    color: blue;
}
```

- class name only to a specific tag (no spaces)

Simple Selectors – Multiple Different Selectors

HTML

```
<div id="title">
    ...
</div>
<div class="thread">
    ...
</div>
<p>
    Hello
</p>
```

CSS

```
#title, .thread, p {
    background-color:black;
}
```

- multiple, different simple selectors separated by comma (,)

Combinator Selectors – Descendant Selector

HTML

```
<div>
  <h2> Header </h2>
  <p> Text1 </p>
  <b>
    <p> Text2 </p>
  </b>
</div>
```

CSS

```
div p {
  color: blue;
}
```

- all occurrences of p inside div elements, regardless of levels

Combinator Selectors – Child Selector

HTML

```
<div>
  <p> Text1 </p>
  <b>
    <p> Text2 </p>
  </b>
  <p> Text3 </p>
</div>
```

CSS

```
div > p {
  color: blue;
}
```

- all occurrences of p with div directly as parent, denoted by >

Combinator Selectors – Adjacent Sibling Selector

HTML

```
<h1> Title </h1>  
<p> Some Text </p>  
<h2> Subtitle </h2>  
<p> Some new text </p>
```

CSS

```
h1 + p {  
    color: silver;  
}
```

- all occurrences of p right after h1 on the same level denoted by +

Combinator Selectors – General Sibling Selector

HTML

```
<p> Alternative text </p>
<h1> Title </h1>
<p> Some Text </p>
<h2> Code and Text </h2>
<div id="hello">
    <p> hello </p>
</div>
<pre> some code </pre>
<p> Some new text </p>
```

CSS

```
h1 ~ p {
    color: green;
}
```

- all occurrences of p anywhere after h1 on the same level denoted by ~

Pseudo-class Selectors

HTML

```
<a href="#top">  
    Some link  
</a>
```

CSS

```
a:hover {  
    color: orange;  
}  
  
a:visited {  
    color: red;  
}
```

- a special state of an element denoted by colon (:

Pseudo-element Selectors

HTML

```
<p>
```

```
  This is one very long paragraph which  
  can span multiple lines in the screen.  
  The rest will not be styled.
```

```
</p>
```

CSS

```
p::first-line{  
    color: green;  
}
```

- a special part of an element denoted by double colon (::)

Attribute Selector 1

HTML

```
<p id="title">
    Some title
</p>
<p id="title-two">
    Another title
</p>
<p class="comment">
    Some text
</p>
```

CSS

```
p[id] {
    font-weight: bold;
}
```

- all elements with the target attribute written in side []

Attribute Selector 2

HTML

```
<p id="title">
    Some title
</p>
<p id="content">
    Some text
</p>
```

CSS

```
p[id="content"] {
    font-weight: bold;
}
```

- all elements with the **exact** value in the target attribute written in side []

Attribute Selector 3

HTML

```
<div id="menu">
    ...
</div>
<div id="list-groups">
    ...
</div>
<div id="list-groups">
    ...
</div>
```

CSS

```
div[id^="list"] {
    background-color:black;
}
```

- all elements with the target attribute **starting** with a substring

Attribute Selector 4

HTML

```
<div id="names">
    ...
</div>
<div id="addresses">
    ...
</div>
<div id="footer">
    ...
</div>
```

CSS

```
div[id$="es"] {
    background-color:black;
}
```

- all elements with the target attribute **ending** with a substring

Attribute Selector 5

HTML

```
<div id="lastname">
    ...
</div>
<div id="nickname">
    ...
</div>
<div id="footer">
    ...
</div>
```

CSS

```
div[id*="na"] {
    background-color:black;
}
```

- all elements with the target attribute **containing** a substring

Summary of Selectors

Symbol	Selector	Purpose
	Simple tag	Selects given tags, IDs (#), or class names (.)
a, b	Multiple element	Selects multiple elements
a b	Descendant	Selects all b inside a
a > b	Child	Selects all b with a directly as parent
a + b	Adjacent sibling	Selects all b right after a given same level
a ~ b	General sibling	Selects all b anywhere after a given same level
a:b	Pseudo-class	Selects special state
a::b	Pseudo-element	Selects special element
[a]	Attribute	Selects given an attribute (=, ^=, \$=, *=)

CSS Specificity

order / hierarchy of the CSS rules

CSS Specificity

The styling declared in CSS ***cascade*** depending on the **source** (*where it was declared*) and the **selector** used.

```
<div id="a-02"> Hello </div>
```

- A. `div#a-02 { color: blue }`
- B. `div[id="a-02"] { color: red }`

Guess

- Which do you think will be applied?

Specificity Hierarchy Groups

Selector	Type	Definition	Points
style=""	Inline styles	Anything placed in the style attribute of the element has precedence over anything.	1,000
#id	IDs	Unique identifier for an element	100
.class	Classes, attributes, and pseudo-classes	Includes .class, [attributes], :hover, :focus, etc.	10
element	Elements and pseudo-elements	For example ::before, ::after, ::first-line	1

Scoring



inline	#id	.class	element
0	0	0	0
0	0	0	1
0	0	0	2
0	0	0	2
0	0	0	3

Scoring

h1 + *[rel=up]

ul ol li.red

li.red.level

style=""

div p

inline	#id	.class	element
0	0	10	1
0	0	10	3
0	0	20	1
1000	0	0	0
0	0	0	2

Scoring

`div p.special`

`#sith`

`body #id .sith p`

`#p1 #p2 p.cool`

`p:not(.cool)`

*:not() is a pseudo
class with no points!*

inline	#id	.class	element
0	0	10	2
0	100	0	0
0	100	10	2
0	200	10	1
0	0	10	1

CSS Specificity

The styling declared in CSS ***cascade*** depending on the **source** (*where it was declared*) and the **selector** used.

```
<div id="a-02"> Hello </div>
```

- A. `div#a-02 { color: blue; }` **1 + 100**
- B. `div[id="a-02"] { color: red; }` **1 + 10**

Recitation

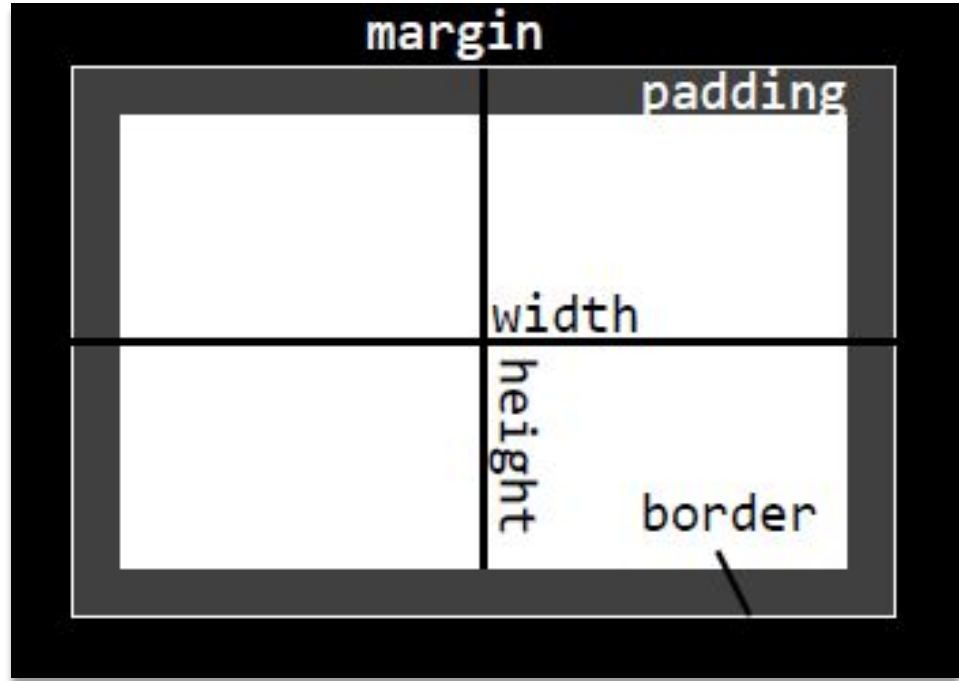
- What is the score for A?
- What is the score for B?

CSS Properties

on layouts, texts, lists, tables

Basic Box Model

- height
- width
- float
- clear
- border
- padding
- margin



The CSS basic box model

All elements are represented as a rectangular box following the basic box model.

height and width

- **auto** - will be browser calculated
- **absolute values**
 - px (relative to viewing device's pixel density),
 - cm, mm, in, pt (1/72 of in), pc (12 pt)
- **relative values**
 - em (relative to font-size of the element; 2em means 2x font size)
 - ch (relative to the width of "O")
 - rem (relative to font-size of the root element)
 - vw, vh, vmin, vmax (relative to viewport dimensions)
 - % (relative to the size of the containing block)
- **inherit** – Inherits the property from the parent element

min- and max-width

min-width

- Sets the **minimum width** of an element
- Will be applied if the content is smaller than the minimum width, else it has no effect

max-width

- Sets the **maximum width** of an element
- Will be applied if the content is larger than the maximum width, else it has no effect

min- and max-height

min-height

- Sets the **minimum height** of an element.
- Will be applied if the content is smaller than the minimum height, else it has no effect

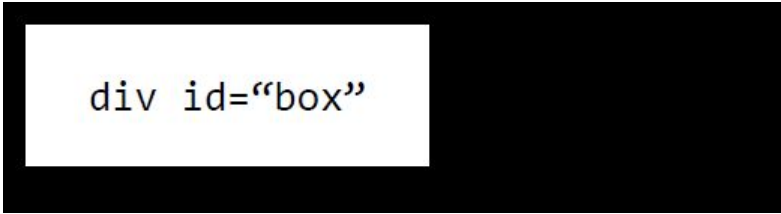
max-height

- Sets the **maximum height** of an element.
- Will be applied if the content is larger than the maximum height, else it has no effect

float

- can be ***left / right / none / inherit***
- makes elements float to the *right* or *left* of the screen, positioned where they are in the HTML
- 'float'-ing an element essentially removes it from the normal flow of the layout, but other elements would still flow around it

```
#box {  
  float: left;  
  margin-right: 10px;  
}
```



div id="box"

clear

- specifies whether it should be rendered below a preceding floated element
 - none – default. Allows floating elements on both sides
 - left – no floating elements allowed on the left side
 - right – no floating elements allowed on the right side
 - both – no floating elements allowed on either side
 - inherit – value will be inherited from parent

```
#box1 { float: left; }
```

```
#box2 { float: left; }
```

```
#box3 { clear: left; } /* also works for clear: both; */
```

```
div id="box1"
```

```
div id="box2"
```

```
div id="box3"
```

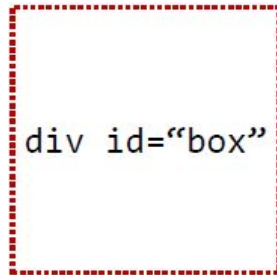
border

- specifies the style, width, and color of an element's border
- border-top, border-bottom, border-left, border-right can be used to style each border line manually

```
#box { border: 1px dotted red; } /* shorthand */
```

```
/* separate */
```

```
#box {  
    border-width: 1px;  
    border-style: dotted;  
    border-color: red;  
}
```



padding

- generates space around an element's content, inside of any defined borders
- padding-top, padding-left, padding-right, padding-bottom can be used to adjust each padding manually

```
#box { padding: 5px; }
```

```
/* (top & bottom, right & left) */
```

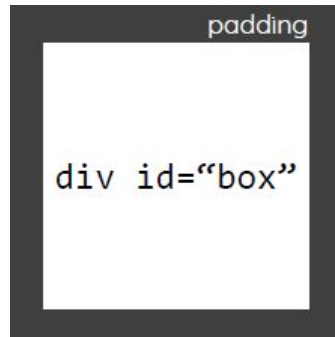
```
#box { padding: 5px 5px; }
```

```
/* (top, right & left, bottom) */
```

```
#box { padding: 5px 5px 5px; }
```

```
/* (top, right, bottom, left) */
```

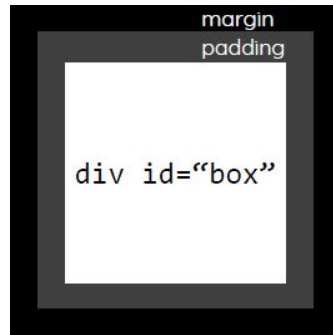
```
#box { padding: 5px 5px 5px 5px; }
```



margin

- create space around elements, outside of any defined borders
- margin-top, margin-left, margin-right, margin-bottom can be used to adjust each margin manually

```
#box {  
    margin: 5px;  
}
```



position

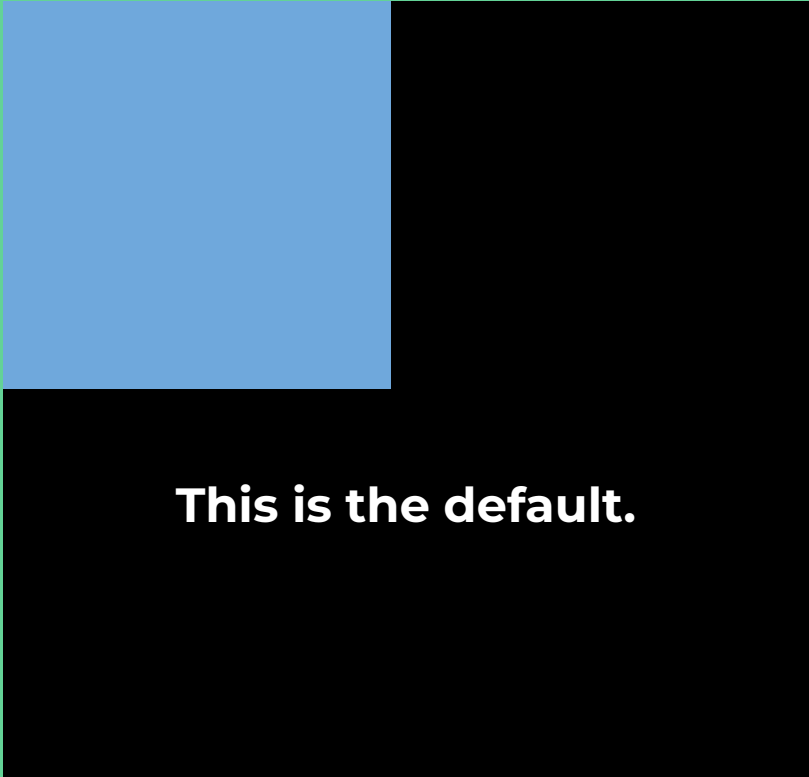
- sets how an element is positioned in a document. Possible values are:
 - static
 - relative
 - absolute
 - fixed
 - sticky

Related properties:

top, **left**, **right**, and **bottom** – determines the final location of the element. May behave differently based on the value of **position**.

static

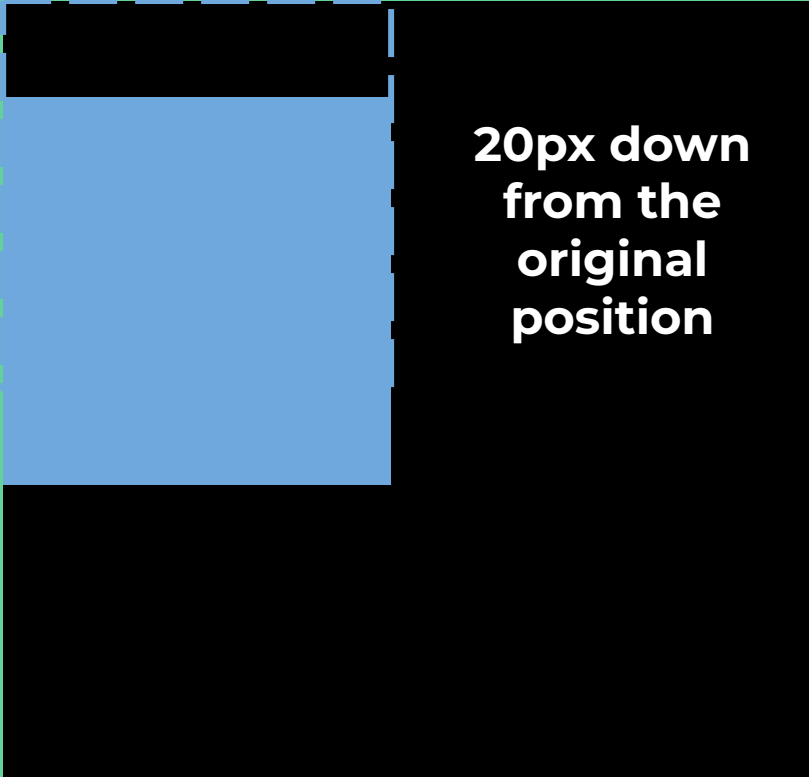
```
#blue-div {  
  position: static;  
  width: 100px;  
  height: 100px;  
}
```



This is the default.

relative

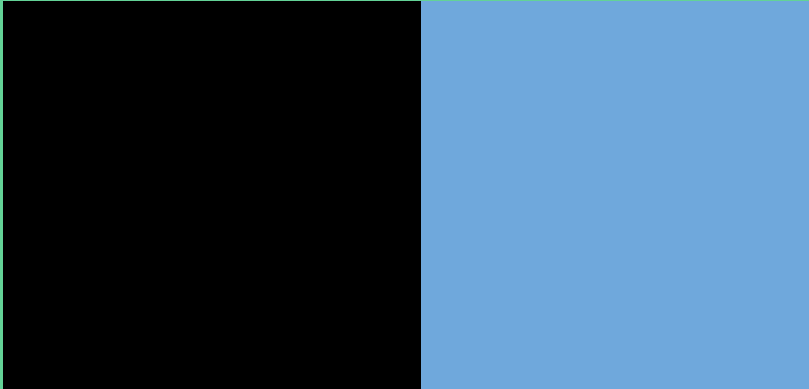
```
#blue-div {  
  position: relative;  
  width: 100px;  
  height: 100px;  
  top: 20px;  
}
```



20px down
from the
original
position

absolute

```
#blue-div {  
  position: absolute;  
  width: 100px;  
  height: 100px;  
  right: 0px;  
}
```

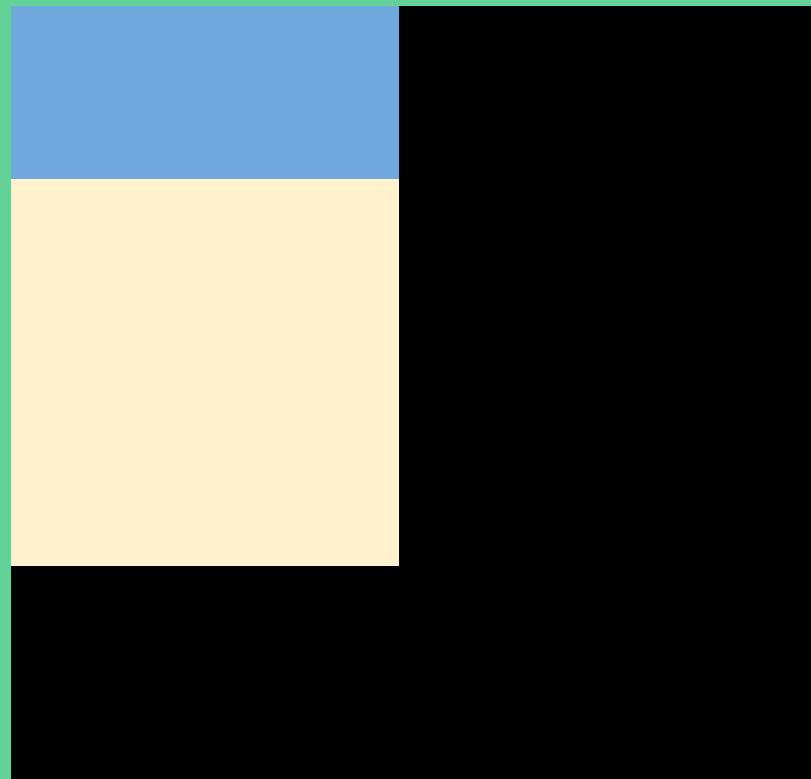


**Removes the div from the
order of things and places
it exactly where it is
specified to go.**

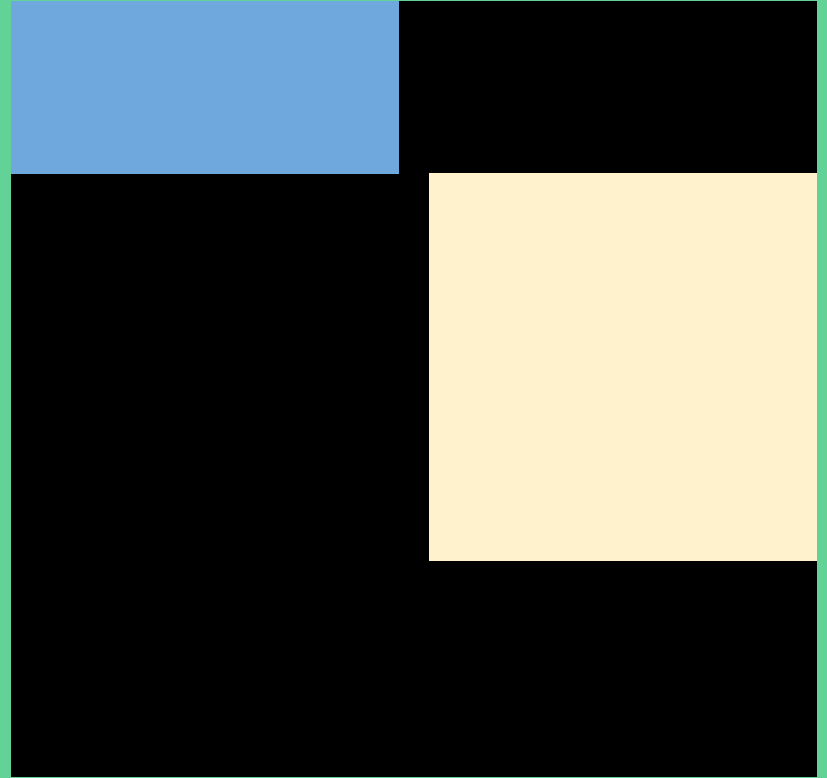
Note: **float** and **clear** will not work
properly with absolute


```
#blue-div {  
  width: 100px;  
  height: 30px;  
  right: 0px; <* no effect since not positioned*>  
}
```

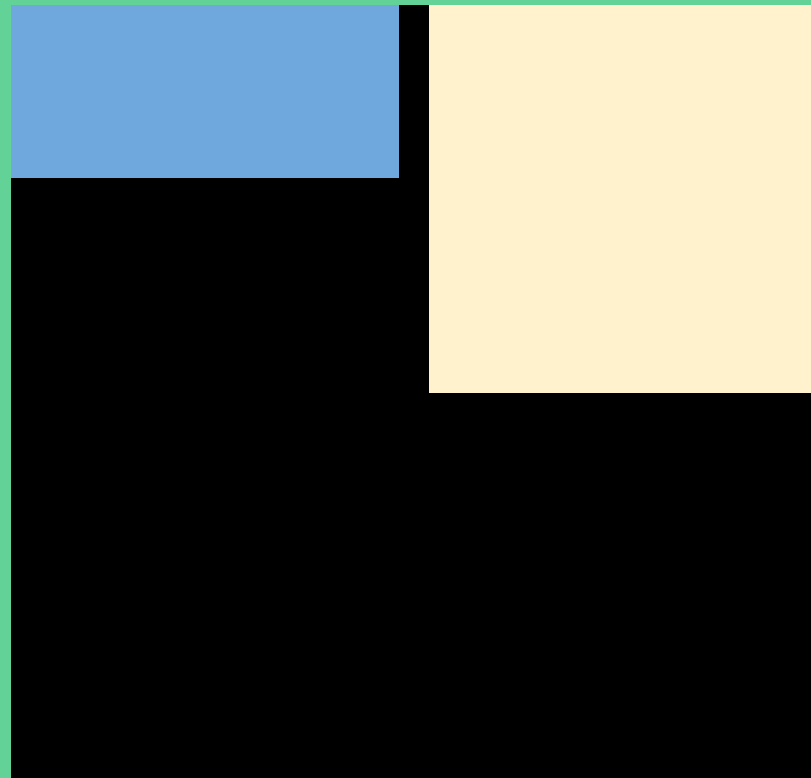
```
#yellow-div {  
  width: 100px;  
  height: 100px;  
}
```



```
...  
#yellow-div {  
  width: 100px;  
  height: 100px;  
  position: absolute;  
  right: 0px;  
}
```



```
...  
#yellow-div {  
  width: 100px;  
  height: 100px;  
  position: absolute;  
  right: 0px;  
  top: 0px;  
}
```



fixed

```
#blue-div {  
  position: fixed;  
  width: 100px;  
  height: 10px;  
  bottom: 0;  
  right: 0;  
}
```

Position is relative to the viewport, so it stays in the same spot even if the page is scrolled.

sticky

```
#blue-div {  
  position: sticky;  
  width: 200px;  
  height: 10px;  
  top: 0px;  
}
```

I am a paragraph placed above this sticky #greendiv....

More text after the #greendiv!
There's a bunch more stuff down here that makes the user have to scroll the page until

sticky

```
#blue-div {  
  position: sticky;  
  width: 200px;  
  height: 10px;  
  top: 0px;  
}
```

More text after the #greendiv!
There's a bunch more stuff down here that makes the user have to scroll the page until everything else can be seen.

One more paragraph here.

sticky

```
#blue-div {  
  position: sticky;  
  width: 200px;  
  height: 10px;  
  top: 0px;  
}
```

There's a bunch more stuff down here that makes the user have to scroll the page until everything else can be seen.

One more paragraph here.

element will **stick** on the screen - useful for navigation headers

display

- sets whether an element is to be treated as a block or an inline element, and the layout used for its children as well

Value	Description
inline	Displays an element as an inline element
block	Displays an element as a block element
contents	Makes the container disappear, making its child elements children of the element the next level up in the DOM
flex	Displays an element as a block-level flex container
grid	Displays an element as a block-level grid container
inline-block	Displays an element as an inline-level block container. The element itself is formatted as an inline element, but you can apply height, width, padding, and margin values
none	The element is completely hidden from the document flow (does not take up any space)

3 ways to Responsive Design - Flexbox

```
1 <!DOCTYPE html>
2 <html lang="en">
3   <head>
4     <title>Flexbox</title>
5     <style>
6       #container {
7         display: flex;
8         flex-wrap: wrap;
9       }
10      #container > div {
11        color: white;
12        background-color: green;
13        font-size: 20px;
14        margin: 20px;
15        padding: 20px;
16        width: 200px;
17      }
18    </style>
19  </head>
20  <body>
21    <div id="container">
22      <div>Some text 1!</div>
23      <div>Some text 2!</div>
24      <div>Some text 3!</div>
25      <div>Some text 4!</div>
26      <div>Some text 5!</div>
27      <div>Some text 6!</div>
28      <div>Some text 7!</div>
29      <div>Some text 8!</div>
30      <div>Some text 9!</div>
31      <div>Some text 10!</div>
32      <div>Some text 11!</div>
33      <div>Some text 12!</div>
34    </div>
35  </body>
36 </html>
```

3 ways to Responsive Design - Grid

```
1 <!DOCTYPE html>
2 <html lang="en">
3   <head>
4     <title>Grid</title>
5     <meta name="viewport" content="width=device-width, initial-scale=1.0">
6     <style>
7       .grid {
8         background-color: green;
9         display: grid;
10        padding: 20px;
11        grid-column-gap: 20px;
12        grid-row-gap: 10px;
13        grid-template-columns: 200px 200px auto;
14      }
15      .grid-item {
16        background-color: white;
17        font-size: 20px;
18        padding: 20px;
19        text-align: center;
20      }
21    </style>
22  </head>
23  <body>
24    <div class="grid">
25      <div class="grid-item">1</div>
26      <div class="grid-item">2</div>
27      <div class="grid-item">3</div>
28      <div class="grid-item">4</div>
29      <div class="grid-item">5</div>
30      <div class="grid-item">6</div>
31      <div class="grid-item">7</div>
32      <div class="grid-item">8</div>
33      <div class="grid-item">9</div>
34    </div>
35  </body>
36 </html>
```

3 ways to Responsive Design - Media Queries

```
1  <!DOCTYPE html>
2  <html lang="en">
3  <head>
4      <title>Media Queries</title>
5      <style>
6          @media (min-width: 600px) {
7              body {
8                  background-color: lightgreen;
9              }
10         }
11         @media (max-width: 599px) {
12             body {
13                 background-color: lightblue;
14             }
15             h1 {color: blue;}
16         }
17     </style>
18 </head>
19 <body>
20     <h1>Welcome to the page!</h1>
21 </body>
22 </html>
```

Text

- With CSS Text, it is possible to style the:
 - color / background color (color and background-color)
 - alignment (text-align, text-align-last, direction, etc.)
 - decoration (text-decoration, text-emphasis, text-shadow, etc.)
 - transform (Using text-transform: uppercase/lowercase, language-specific case mapping, etc.)
 - spacing (text-indent, letter-spacing, line-height, white-space, etc.)
 - *etc.*
- Possible **color** value formats:
 - name – specify a color name like “blue”
 - hex – specify a hex value like “#0000ff”
 - RGB – specify an RGB value like “rgb(0, 0, 255)”

Fonts

- With CSS Font, it is possible to style the:
 - font-family
 - font-size
 - font-stretch
 - font-style
 - font-weight
 - etc.
- You can use a single property, **font**, which is shorthand property for the other different properties

CSS Fonts:

https://developer.mozilla.org/en-US/docs/Web/CSS/CSS_Fonts

https://www.w3schools.com/css/css_font.asp

font syntax:

<https://developer.mozilla.org/en-US/docs/Web/CSS/font#syntax>

Lists

- With CSS lists, you can style the:
 - styling for text also applies
 - bullets styles (basic and custom)
 - padding
 - background
 - *etc.*
- You can also use a shorthand property, **list-style**, for bullet- or numbering-related styling

Tables

- With CSS tables, you can style the:
 - texts and fonts also apply
 - border
 - padding
 - background
 - *etc.*

Variables

- To declare a variable in a **global scope**:

```
:root { /* :root is a pseudo-class which is the root of the HTML document */  
  --primary: #008688; /* - indicates a variable is being declared */  
  --accent: #64cacc;  
}
```

- For **local scope**, declare the variable inside the selector
- To use the variable:

```
var(--primary)
```

- For example:

```
color: var(--primary);
```


Animations

- CSS Animations are basically transition from one style to another
- You can also specify style keyframes

Activity

- [HO2] CSS
 - HO2 is due **May 21 (Thursday)**

NOTES...

- CCDEVAP is still online until June 1
- Our first f2f meeting will be on June 4
- On Thursday **May 21**,
 - Sharing of beautified personal pages with class
 - MP groupings
 - Creating project proposals (MP) - due on **May 25 (Monday)**